6.3.2 Phenomena

For reasonable transaction throughput values, it makes sense to imply transaction *serializability*. As the incoming traffic grows, the price for strict data integrity becomes too high, and this is the primary reason for having multiple isolation levels. Relaxing serializability guarantees may generate data integrity anomalies, which are also referred as *phenomena*.

The SQL-92 standard introduced three phenomena that can occur when moving away from a serializable transaction schedule:

- *dirty read*
- *non-repeatable read*
- *phantom read.*

In reality, there are other phenomena that can occur due to transaction interleaving, as the famous paper *A Critique of ANSI SQL Isolation Levels*[4] describes:

- *dirty write*
- *read skew*
- *write skew*
- *lost update.*

> Choosing a certain isolation level is a trade-off between increasing concurrency and acknowledging the possible anomalies that might occur.
>
> Scalability is undermined by contention and coherency costs. The lower the isolation level, the less locking (or multi-version transaction abortions), and the more scalable the application gets.
>
> However, a lower isolation level allows more phenomena, and the data integrity responsibility is shifted from the database side to the application logic, which must ensure that it takes all measures to prevent or mitigate any such data anomaly.

Before jumping to isolation levels, it is better to understand what's behind each particular phenomenon and how it can affect data integrity. When choosing a given transaction isolation level, understanding phenomena becomes fundamental to taking the right decision,

[4] http://research.microsoft.com/apps/pubs/default.aspx?id=69541

6.3.2.1 Dirty write

A dirty write happens when two concurrent transactions are allowed to modify the same row at the same time. As previously mentioned, all changes are applied to the actual database object structures, which means that the second transaction simply overwrites the first transaction pending change.

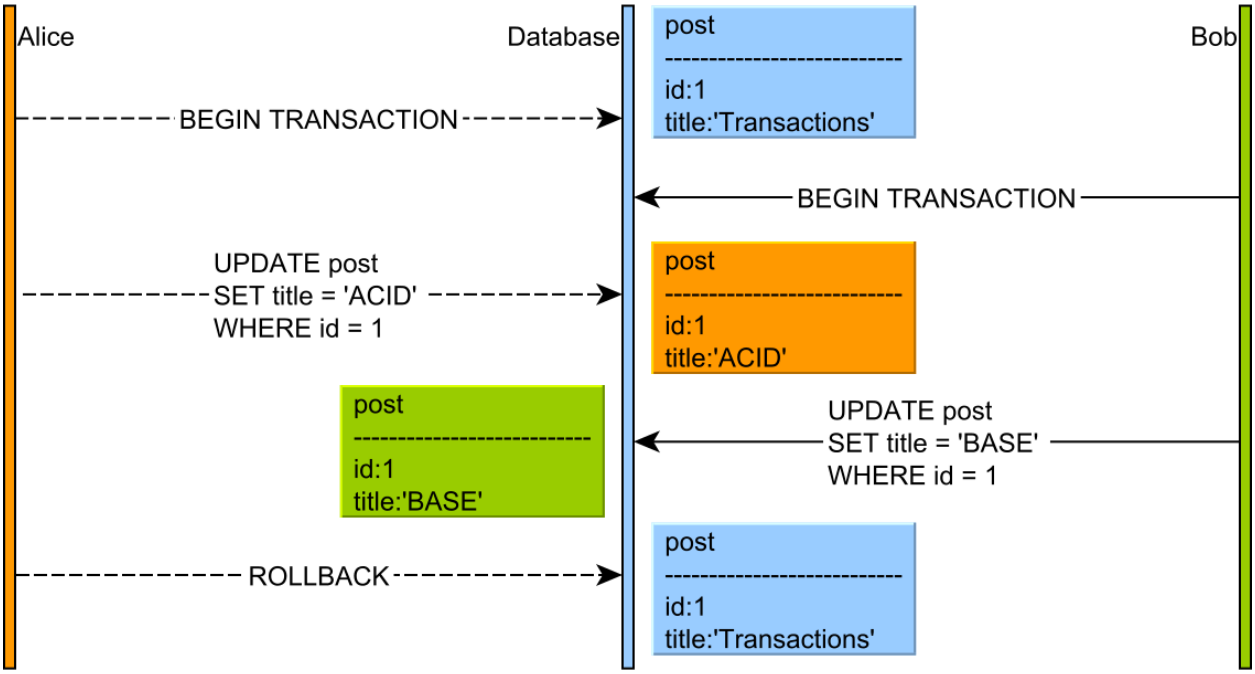

Figure 6.4: Dirty write

If the two transactions commit, one transaction will silently overwrite the other transaction, causing a *lost update*. Another problem arises when the first transaction wants to roll back. The database engine would have to choose one of the following action paths:

- It can restore the row to its previous version (as it was before the first transaction changed it), but then it overwrites the second transaction uncommitted change.
- It can acknowledge the existence of a newer version (issued by the second transaction), but then, if the second transaction has to roll back, its previous version will become the uncommitted change of the first transaction.

If the database engine did not prevent dirty writes, guaranteeing rollbacks would not be possible. Because atomicity cannot be implemented in the absence of reliable rollbacks, all database systems must, therefore, prevent dirty writes.

Although the SQL standard does not mention this phenomenon, even the lowest isolation level (*Read Uncommitted*) can prevent it.

6.3.2.2 Dirty read

As previously mentioned, all database changes are applied to the actual data structures (memory buffers, data blocks, indexes). A dirty read happens when a transaction is allowed to read the uncommitted changes of some other concurrent transaction. Taking a business decision on a value that has not been committed is risky because uncommitted changes might get rolled back.

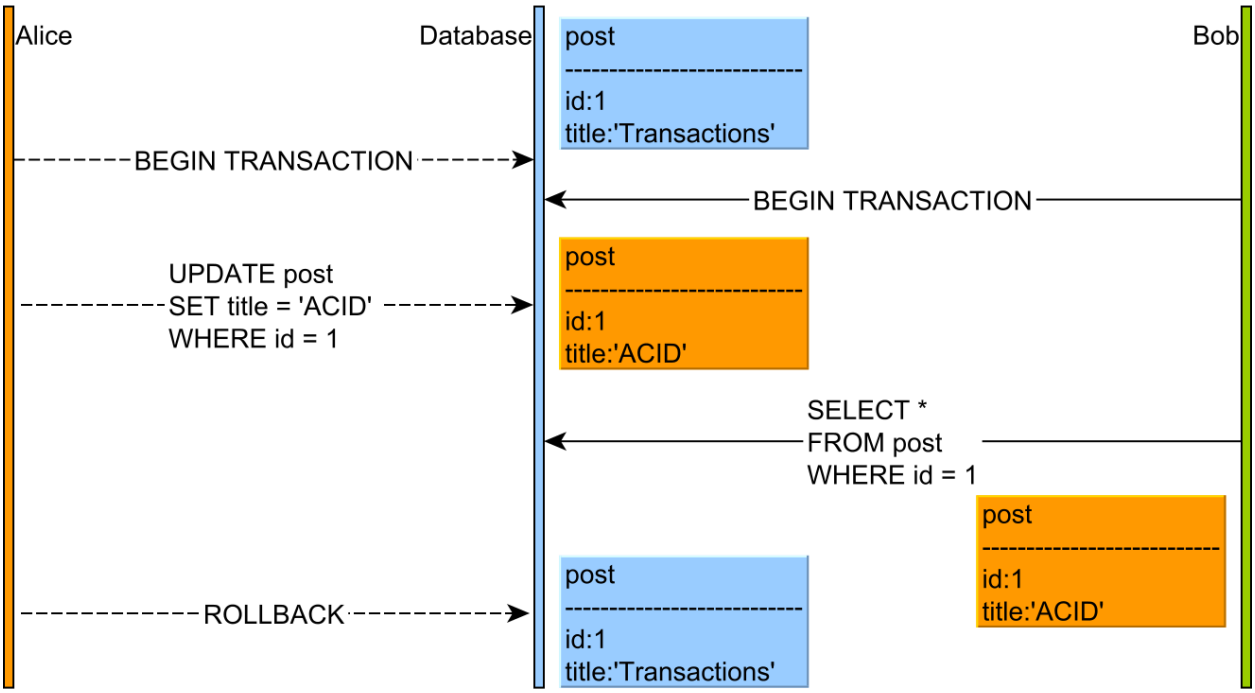

Figure 6.5: Dirty read

This anomaly is only permitted by the Read Uncommitted isolation level, and, because of the serious impact on data integrity, most database systems offer a higher default isolation level.

To prevent dirty reads, the database engine must hide uncommitted changes from all other concurrent transactions. Each transaction is allowed to see its own changes because otherwise the read-your-own-writes consistency guarantee is compromised.

If the underlying database uses 2PL (Two-Phase Locking), the uncommitted rows are protected by write locks which prevent other concurrent transactions from reading these records until they are committed.

When the underlying database uses MVCC (Multi-Version Concurrency Control), the database engine can use the undo log which already captures the previous version of every uncommitted record, to restore the previous value in other concurrent transaction queries. Most database systems optimize the before image restoring process, therefore lowering its overhead on the overall application performance.

Read Uncommitted is rarely needed (non-strict reporting queries where dirty reads are acceptable), so Read Committed is usually the lowest practical isolation level.

6.3.2.3 Non-repeatable read

If one transaction reads a database row without applying a shared lock on the newly fetched record, then a concurrent transaction might change this row before the first transaction has ended.

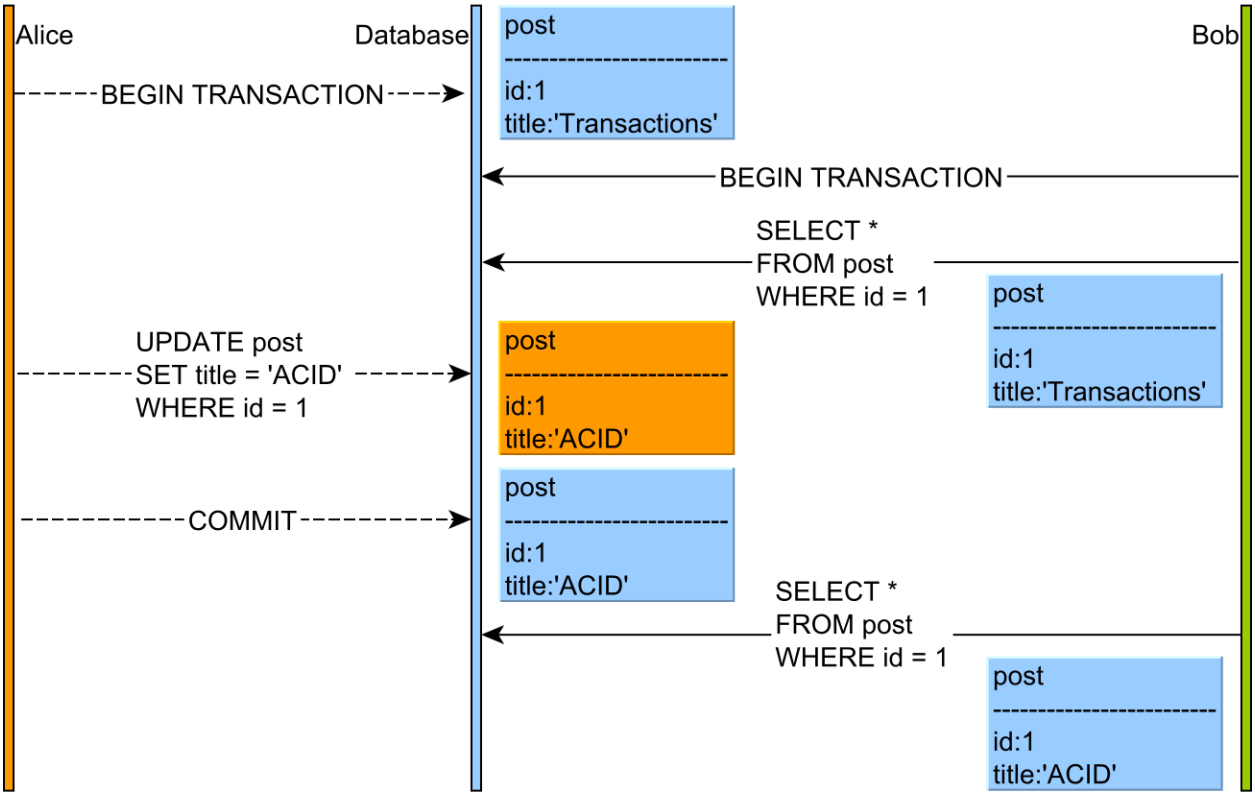

Figure 6.6: Non-repeatable read

This phenomenon is problematic when the current transaction makes a business decision based on the first value of the given database row (a client might order a product based on a stock quantity value that is no longer a positive integer).

Most database systems have moved to a Multi-Version Concurrency Control model, and shared locks are no longer mandatory for preventing non-repeatable reads. By verifying the current row version, a transaction can be aborted if a previously fetched record has changed in the meanwhile.

Repeatable Read and Serializable prevent this anomaly by default. With Read Committed, it is possible to avoid non-repeatable (fuzzy) reads if the shared locks are acquired explicitly (e.g. `SELECT FOR SHARE`).

Some ORM frameworks (e.g. JPA/Hibernate) offer application-level repeatable reads. The first snapshot of any retrieved entity is cached in the currently running *Persistence Context*. Any successive query returning the same database row is going to use the very same object that was previously cached. This way, the fuzzy reads may be prevented even in Read Committed isolation level.

6.3.2.4 Phantom read

If a transaction makes a business decision based on a set of rows satisfying a given predicate, without range locks, a concurrent transaction might insert a record matching that particular predicate.

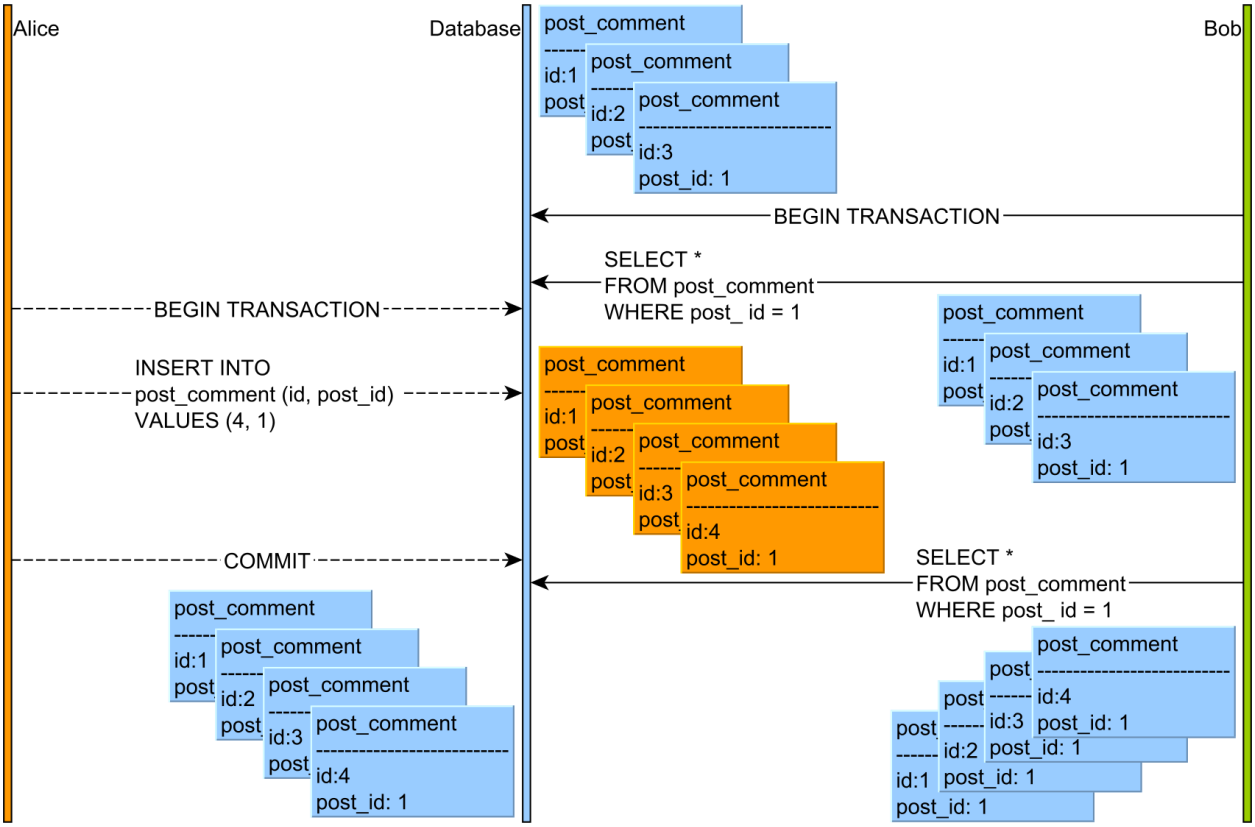

Figure 6.7: Phantom read

The SQL standard says that Phantom Read occurs if two consecutive query executions render different results because a concurrent transaction has modified the range of records in between the two calls. Although providing consistent reads is a mandatory requirement for serializability, that is not sufficient. For instance, one buyer might purchase a product without being aware of a better offer that was added right after the user has finished fetching the offer list.

The 2PL-based Serializable isolation prevents Phantom Reads through the use of predicate locking. On the other hand, MVCC database engines address the Phantom Read anomaly by returning consistent snapshots. However, a concurrent transaction can still modify the range of records that was read previously. Even if the MVCC database engine introspects the transaction schedule, the outcome is not always the same as a 2PL-based implementation. One such example is when the second transaction issues an insert without reading the same range of records as the first transaction. In this particular use case, some MVCC database engines will not end up rolling back the first transaction.